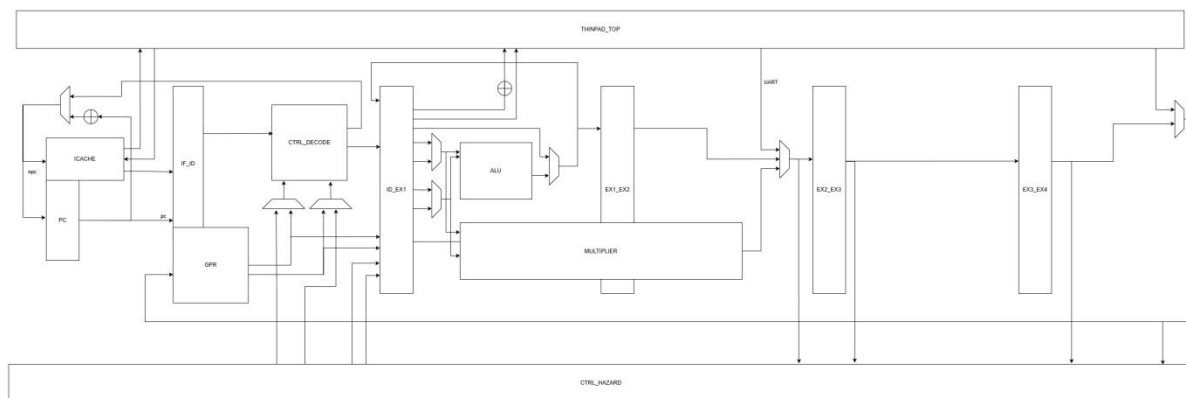


NSCSCC2025 设计报告

西北工业大学 吴语

一、龙芯杯个人赛设计概述：

本设计基于龙芯杯 MIPS 指令系统规范，采用哈佛架构，顺序单发射静态七级流水线实现了手册上要求指令，访存使用三个流水级别，总体架构分别是预取指，取指，译码，EX1，EX2，EX3，EX4/预写回，目前为 160MHz。使用了三级旁路前递与阻塞解决数据冒险与结构冒险，完成了串口通信并且于在线平台完成了测试。EXE 级使用了 booth2 算法的乘法器。



PC: pc 寄存器

ICACHE: 指令缓存 采用 block ram

IF_ID, ID_EX1, EX1_EX2 ……: 流水线寄存器

CTRL_DECODE: 译码器,同时完成跳转指令的判断和计算

GPR: 32 位通用寄存器, 采用 block ram 并且构建了旁路

ALU: 除乘法外的计算单元

MULTIPLIER: booth2 算法+csa 压缩乘法器, 两周期

CTRL_HAZARD: 阻塞与前递信号控制

THINPAD_TOP: 通讯控制,完成串口控制以及和板子 SRAM 通信信号

二、设计方案：

总体设计思路

本作品主要由 thinpad_topd 顶层-cpu 处理器构成，其他所有模块均在 cpu 中。

设计架构：

thinpad_top.v //顶层文件(含通讯控制(内存映射+串口))

```
└─ ddd_cpu.v //cpu 顶层文件
    └─ ctrl_decode.v //指令译码
    └─ icache.v //指令缓存
    └─ ctrl_hazard.v //冒险前递与阻塞
    └─ regfile.v //通用寄存器(tool.v 中)
    └─ pc.v //PC 寄存器(tool.v 中)
    └─ multiplier.v //乘法器(tool.v 中)
        └─ csa_adder.v //压缩器(tool.v 中)
```

代码命名规则：一个信号的产生端或最终使用端采用其含义直接命名，如译码模块产生的所有译码信号均为直接含义。直接含义命名（在产生端或最终使用端）信号名应直接描述其功能或作用，避免缩写混淆，如：reg_write（寄存器写使能），d_aluop（ALU 操作码），gpr_wid（通用寄存器写地址）；控制信号分类，以 s_ 开头，表示选择：s_alu_b（选择 ALU 的第二个操作数来源，即立即数）s_lw（Load Word 控制信号）数据类型，使用 _reg、_wire 或 _imm 等后缀区分变量类型：a_reg（寄存器 A 的值）imme（立即数）；流水线变量命名规则：信号的作用域为 IF，ID，EX1，EX2，EX3，EX4 的数据分别为 1，2，3，4，6 作为后缀。

基本模块的实现

1、PC

指令寄存器，同步复位，高电平复位，初地址为 32'h8000_0000，PC 寄存器中存放的是当前正在 IF 的指令地址，npc 是下一条指令的地址。

2、GPR:

寄存器堆，可以实现两读一写，在时钟上升沿同时读写，采用 block ram 作为存储单元，并且额外实现了旁路以解决读写冲突。

3、ctrl_decode 模块

实现译码以及跳转地址的判断与计算。

在这个模块得到立即数偏移，后继的所有基本选择控制信号，如链接操作选择存数为 pc+8，还是 alu 计算得到的数据，或是方存得到的数据等。

跳转判断比较，跳转地址计算是与译码同步进行的。如果判断出跳转，则将已经在 pc 寄存器中的 pc 和取得的指令无效化。

4、alu 模块:

并行完成所有的操作类型，除乘法外，一共有 8 种基本操作。功能为根据译码结果进行指定的计算，给出运算结果、要写入的寄存器地址或访存地址。

5、ctrl_hazard 模块:

这个模块负责控制所有的阻塞与前递信号。数据源通过 EX1, EX2, EX3, EX4 级的写寄存器 id 与 id, exe 级的使用寄存器 id 判断是否发生数据冒险。这里只有跳转前递判断需要判断 EX1 的写寄存器 id。

其中对于 ID 级（判断跳转）的前递为前递到 Ctrl_decode 之前，EX1 级的前递为前递到 EX1 之前的流水线寄存器，通过流水线寄存器完成前递；EX1 级可以算好的数据通过特殊传递（不通过 Ctrl_hazard）传递到 EX1 之前的流水线寄存器，通过流水线寄存器之后的组合选择完成前递。

控制流程：

- (1) 获取有效的 EXE 源寄存器
- (2) 将 EXE 源寄存器与 mem 阶段的写地址比较：
- (3) 获取前递数据

阻塞控制：

(1) 数据冒险阻塞： exe 阶段指令是 lw/lb， id 阶段的源寄存器与 lw/lb 指令要写的目标寄存器相同，必须先阻塞一个时钟周期，再通过 wb2exe 旁路解决该数据冒险。同理， id 需要用于判断跳转的操作源为 exe 的写寄存器或者 mem 级 lw/lb 时也需要阻塞。

d) 总体阻塞设计

所有的阻塞信号分为 flush（刷新为 nop）与 write（!write 为阻塞，保持不变）。

控制信号源为跳转控制/寄存器源冲突等。

6、流水线设计说明

所有的指令分为 4 个类别：

- 1、跳转指令，在 ID 级完成，延迟槽下个周期进入 ID，无刷新操作
- 2、普通运算指令，可以在一个周期内得到结果，在 EX1 级完成
- 3、需要两个周期完成的指令，即乘法指令
- 4、访存指令，所有的访存指令会在 EX1 级发射请求到 thinpad_top.v 中，如果是串口，则会在 EX2 读出，并且结束发送访存请求；普通访存指令持续三个周期发送访存请求级 EX1, EX2, EX3 发送，并在 EX4/pre_WB 级得到，做位选择(如果需要)之后发送到通用寄存器写端口。

7、booth2 乘法器

使用 booth2 位乘+六层华莱士树实现乘法计算，采用 32 位有符号乘法的方式将有/无符号乘法统一计算。

计算的原理是一个数学变形：

$$\begin{aligned}
 B &= -B_{n-1}2^{n-1} + \left(\sum_{i=0}^{n-2} B_i \times 2^i \right) \\
 &= -B_{n-1}2^{n-1} + B_{n-2}2^{n-2} + B_{n-3}2^{n-3} + B_{n-4}2^{n-4} + \dots \\
 &\quad + B_32^3 + B_22^2 + B_12^1 + B_02^0 + B_{-1} \\
 &= (-2B_{n-1} + B_{n-2} + B_{n-3})2^{n-2} + (-2B_{n-3} + B_{n-4} + B_{n-5})2^{n-4} + \dots \\
 &\quad + (-2B_5 + B_4 + B_3)2^4 + (-2B_3 + B_2 + B_1)2^2 + (-2B_1 + B_0 + B_{-1})2^0 \\
 &\text{其中 } B_{-1} = 0
 \end{aligned}$$

将A与B相乘，则：

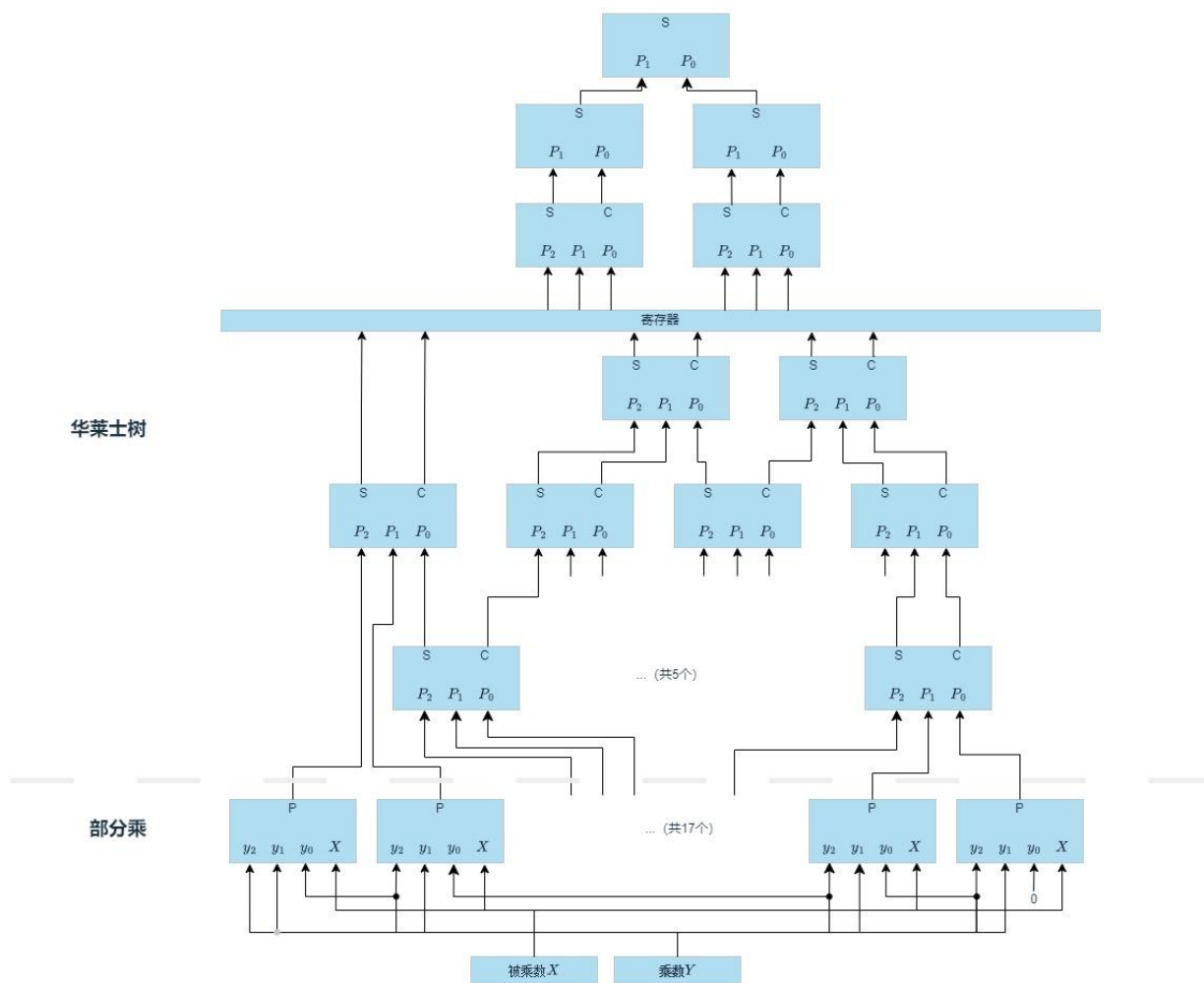
$$\begin{aligned}
 A \times B &= A \times (-2B_{n-1} + B_{n-2} + B_{n-3})2^{n-2} + A \times (-2B_{n-3} + B_{n-4} + B_{n-5})2^{n-4} + \dots \\
 &\quad + A \times (-2B_5 + B_4 + B_3)2^4 + A \times (-2B_3 + B_2 + B_1)2^2 + A \times (-2B_1 + B_0 + B_{-1})2^0
 \end{aligned}$$

B_{i+1}	B_i	B_{i-1}	$-2B_{i+1} + B_i + B_{i-1}$	部分积操作
0	0	0	+0	0
0	0	1	+1	A
0	1	0	+1	A
0	1	1	+2	2A
1	0	0	-2	-2A
1	0	1	-1	-A
1	1	0	-1	-A
1	1	1	-0	0

图4：基4 Booth真值表

将乘法拆分成 16 个部分和，然后通过进位保留加法器 (CSA) 逐步压缩，每个 csa 把三个加数压缩为两个加数。通过六层华莱士树实现十六个部分和的相加。

六层华莱士树如图所示，较图中的结构修改：把寄存器往上移了一层，即第二级只完成两个加法。



CSA (Carry Save Adder) 进位保留加法器：

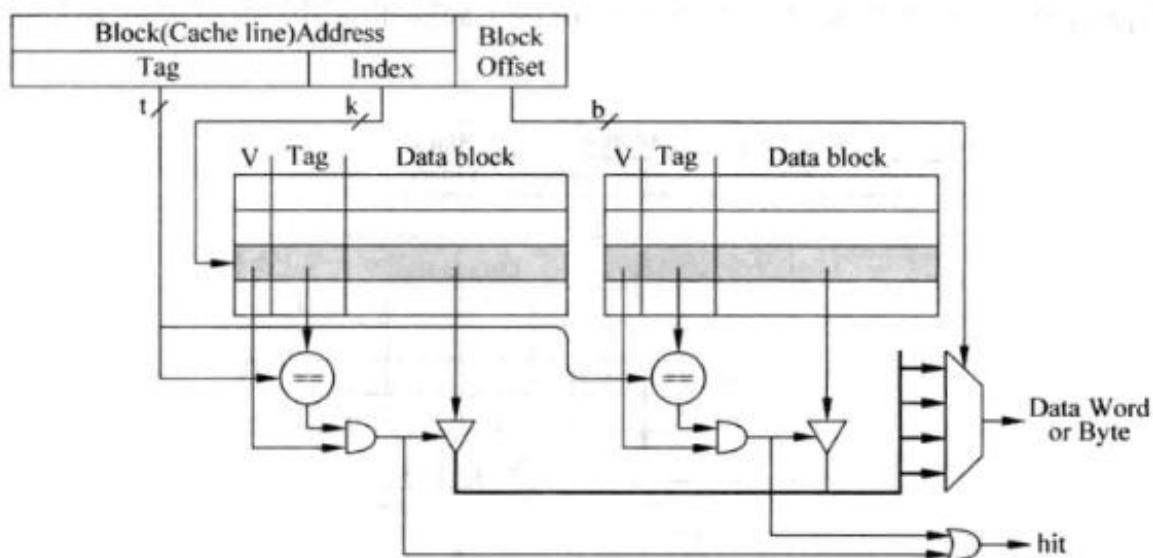
普通加法器：计算进位需要逐级传递，导致关键路径长；

CSA：不立即合并进位，而是将进位 (Carry) 与和 (Sum) 分开保存，直到最后一步才用普通加法器合并。

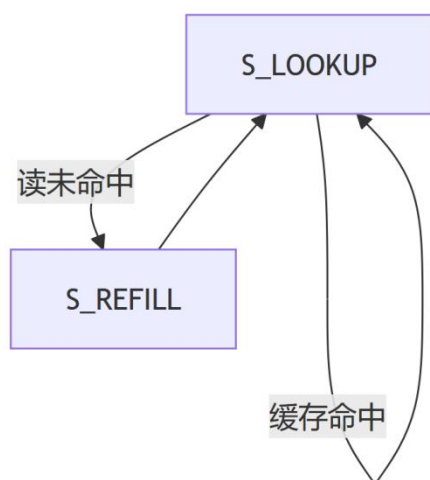
7、指令缓存 icache

结构参考的 CPU 设计实战，主题仍为两路组相连没改变，细节做了部分调整：没有 Dirty 位并且把 Tag 降至 9 位。

组合接受，如果未命中，会在下个周期发送访存请求，三个周期完成一个指令读写。一次未命中填充 4 条指令。如果发生 EX1 级发送访存请求结构冒险，icache 重置请求状态。



状态机如下：



三、设计结果：

1、测试结果

三级测试通过：

最终提交版本

提交截止时间：2025-08-05 23:59:59

最终版本标记时间：2025-07-29 16:19:25

任务 ID	提交时间	版本名称	状态	得分
15694	2025-07-29 16:18:55	09224d1b	Finished	100

最终提交版本

提交截止时间：2025-08-05 23:59:59

最终版本标记时间：2025-07-29 16:19:05

任务 ID	提交时间	版本名称	状态	得分
15670	2025-07-29 15:53:53	09224d1b	Finished	100

性能测试结果 160MHz 0.296s

测试结果



100

nscscc2025_mips_perf 在 FPGA 板 15 11 12 上的结果

```
=== Test STREAM ===
Boot message: 'MONITOR for MIPS32 - initialized.'
User program written
  Program Readback:
    1080043c4080053c3000063c21308600050086100400a5240000828cfcffa2acfi
Program memory content verified
Data memory content verified
Test STREAM run for 0.049s
```

[下载 15 的追踪数据](#)

测试结果



100

nscscc2025_mips_perf 在 FPGA 板 15 11 12 上的结果

```
=== Test MATRIX ===  
Boot message: 'MONITOR for MIPS32 - initialized.'  
User program written  
  Program Readback:  
    4080043c4180053c4280063c60000724251800001a00671080400300405203002:  
Program memory content verified  
Data memory content verified  
Test MATRIX run for 0.067s
```

[下载 11 的追踪数据](#)

测试结果



100

nscscc2025_mips_perf 在 FPGA 板 15 11 12 上的结果

```
=== Test CRYPTONIGHT ===  
Boot message: 'MONITOR for MIPS32 - initialized.'  
User program written  
  Program Readback:  
    4080043cadde053cefbea534cefa063c0cb0c6341000073c25180400251000000:  
Program memory content verified  
Data memory content verified  
Test CRYPTONIGHT run for 0.180s
```

[下载 12 的追踪数据](#)

四、vivado 配置说明

仿真布线策略:

Synthesis	Implementation	Summary	Route Status
Status: Complete	Status: write_bitstream Complete!		
Messages: 137 warnings	Messages: 2 critical warnings		
Active run: synth_2	Active run: 37 warnings		
Part: xc7a200tfg676-2	Active run: impl_3		
Strategy: Flow_PerfThresholdCarry	Part: xc7a200tfg676-2		
Report Strategy: Vivado Synthesis Default Reports	Strategy: Performance_NetDelay_low		
Incremental synthesis: None	Report Strategy: Vivado Implementation Default Reports		
	Incremental implementation: None		

五、参考设计说明

1、串口通讯设计参考:龙芯杯 NSCSCC2020 个人赛开源代码

https://github.com/fluctlight001/cpu_for_nscsc2020

进行了适应性修改,增加了对于 top 文件的接口

2、乘法器设计参考:龙芯杯 NSCSCC2024 团队赛设计文档

<https://library.vincent-ice.me/posts/2024%E9%BE%99%E8%8A%AF%E6%9D%AF%E4%BB%A3%E7%A0%81/>

只参考了乘法器的设计结构,并且做出了调整,调整了中间寄存器的位置。

3、icache 设计参考: 汪文祥,邢金璋. CPU 设计实战. 北京: 机械工业出版社, 2021.